
Thursday, 21st August 2025

# Engineering a Real-Time, Always-On Logging System in D

**Luís Ferreira**

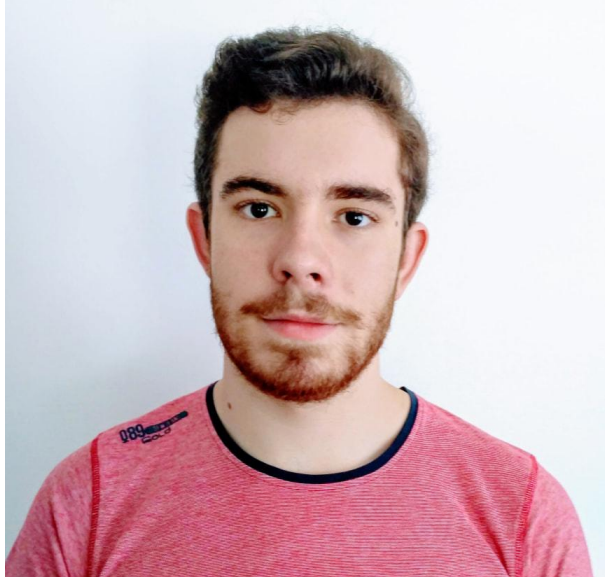
Software Engineer

# Who am I?

I’m a software developer at Weka, with a strong passion for low-level systems programming.

I’m also an open-source contributor, with involvement in the D programming language ecosystem, including contributions to its compiler, standard library, and runtime.

Beyond D, I also made some contributions across other open-source projects such as LLVM, the Linux kernel, and more.



**Luís Ferreira**
Software Engineer

# Why always on logging?

## Why it matters?

On a weka cluster, each node performs several tasks concurrently, sending and receiving data across other nodes, all at a sub millisecond time window.

When issues arise, debugging these tasks requires careful analysis, and in such a complex environment, observability is essential.

Since this mechanism is always enabled, it must be provided through a mechanism that is both fast and efficient.

# Design goals

## Key objectives and requirements

- **Fast to log a single entry:** each log entry should be written with minimal overhead, targeting speeds comparable to a simple *memcpy* operation.

  *Benchmark: ~3.5M entries in ~15ms = ~266M entries/s = ~3.7ns per entry*

- **Fast to read and filter:** given that we are talking about several millions of logs per second, naturally, we easily generate also several gigabytes of traces in a few seconds, so filtering and reading a small portion should be fast.

  *Raw traces are written at +2.3GB/s*

- **Extensive logging and tracing:** Achieving high observability requires collecting tens of thousands of distinct log entries. We aim to go beyond logging by also tracing function calls and returns. This demands support for a wide variety of log structures, each with its own arguments and representations.

  *~$2^{16}$ = ~65 536 unique entries (we might expand to ~$2^{24}$ = 16 777 216 unique entries)*

  *We aim for +95% RCA (**R**oot **C**ause **A**nalysis)*

# Design goals

## Key objectives and requirements

- **Compact log entries:** we want to trace several millions of logs per second from thousands of different log lines and tens of thousands of different function at their *prologue* and *epilogue*, while keeping them very compact, when written.
  *~9 bytes per entry + size of their arguments*
- **User-friendly debugging experience:** reading, filtering and, in general, understanding when and where these logs came from requires user-friendly utilities that facilitate the debugging experience of the issues.

# What is a trace log?

## D API

```
INFO
WARN
ERROR
DEBUG!"this is a debug log that traces val=%s isValid=%s"(val, isValid);
```

The log template, describing the **type** of log used.

The **string** used to format the trace log.

The **arguments** used in the trace log to fill the format string.

# Design structure

## Main components

- **Instrumentor:** executable that pre-process the original source to introduce log entries at function *prologue* and *epilogue*.
- **Writer Library:** core library that contains the routines to write the traces to a shared memory buffer.
- **Dumper:** executable that constantly reads from the shared memory buffer and writes the traces into persistent storage.
- **Daemon:** server that provides several services to read and filter traces from persistent storage.
- **Printer:** non-interactive simpler end-user utility that reads the traces directly from persistent storage and prints the formatted form into the console.
- **Viewer:** interactive utility that serves as a client, used by the end-user to lookup the generated traces and investigate all the logs in an aggregated fashion.

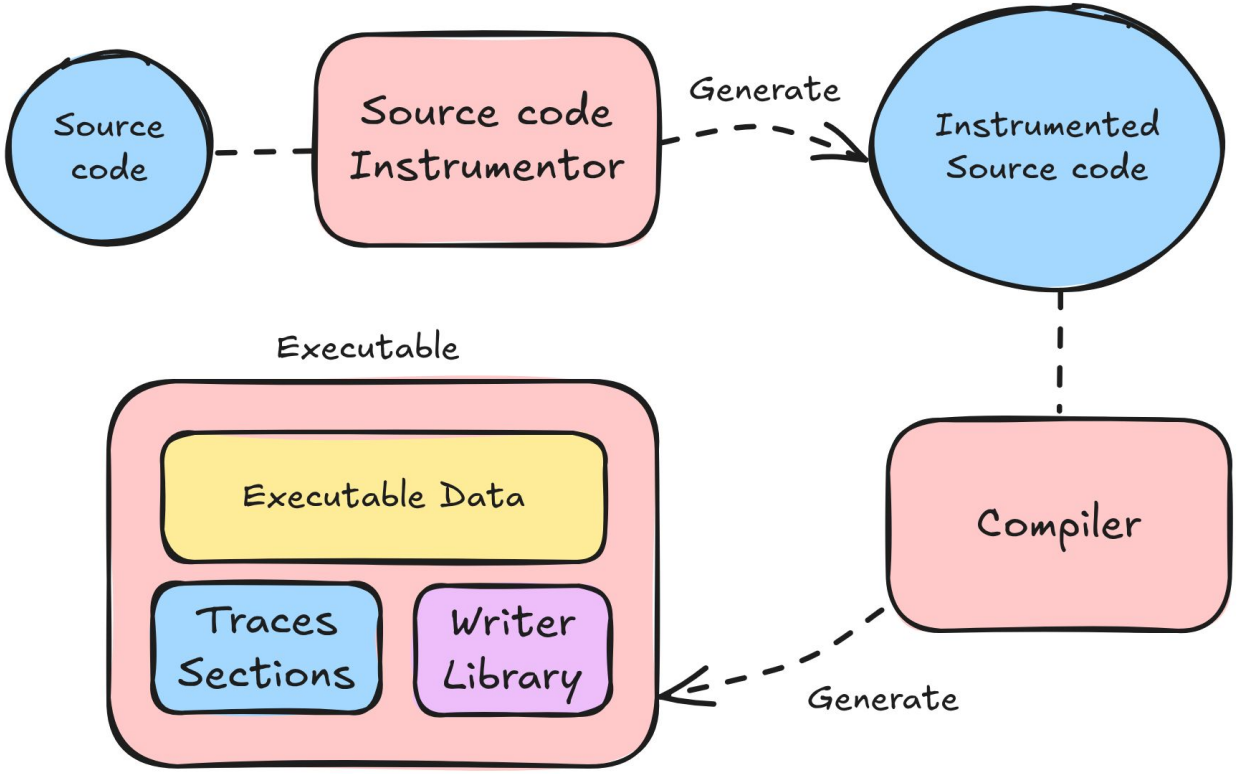
# Design structure

## Producing executable

To generate a binary, we pre-process the D source code using the instrumentor executable, that uses libdparse to generate a resulting instrumented source code with functions hooked with enter/exit trace logs.

Each trace log entry in the source code will generate some metadata in specific binary sections, that are later used to describe a log entry.

The produced binary includes along side the executable data, these generated sections and the traces writing library.






WEKA

# Code Instrumentation

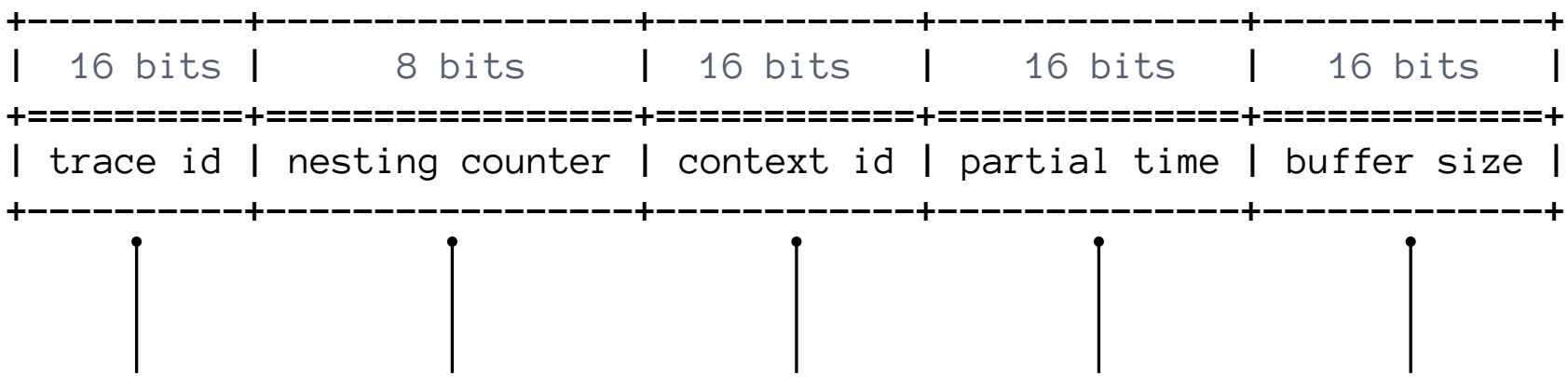
```
auto foobar(string str, bool flag, int value)
{
   // ... code ...
}
```

```
void foobar(string str, bool flag, int value)
{
   FUNC_ENTER!(["str", "flag", "value"])(str, flag, value);
   void __traced_foobar()()
   {
       // ... code ...
   }

   try
   {
       return FUNC_RETURN(__traced_foobar());
   }
   catch (Throwable t)
   {
       FUNC_THROW(t);
       throw t;
   }
}
```

# What is a trace log?

## Entry Header structure




Identifier that **describes** the immutable metadata part of the trace.

**Counter** that identifies in which stack frame **depth** this trace log is nested.

**Context** identifier that tells in which **context** (thread id / fiber id) this trace is running at.

**Field** that contains a **partial diff** of a monotonic **timestamp** counter relative to the previous trace log.

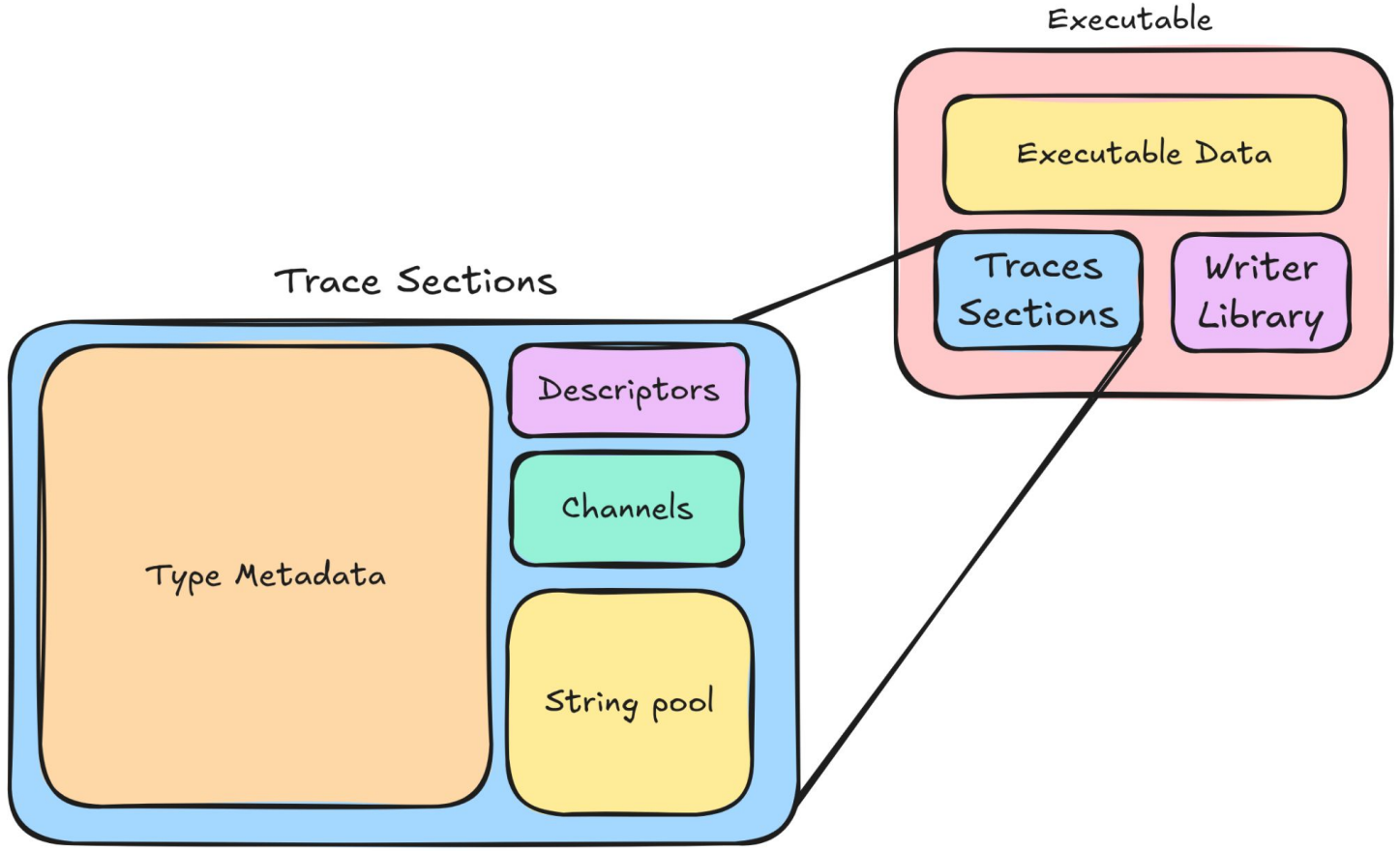
Contains the **size** in **bytes** of the suffixed **buffer** and tells how many bytes to **offset** to the **next entry**.

# What defines a trace?

## Internal sections structure

Using several compiler and linker techniques we can embed custom data as symbols into custom sections in the executable.

In LDC, we use something like `@section("wtracer_descriptor")` built-in attribute to specify the section we want to keep the data, and in some situations the `@assumeUsed` attribute to avoid the linker to eliminate the apparent unused symbol.





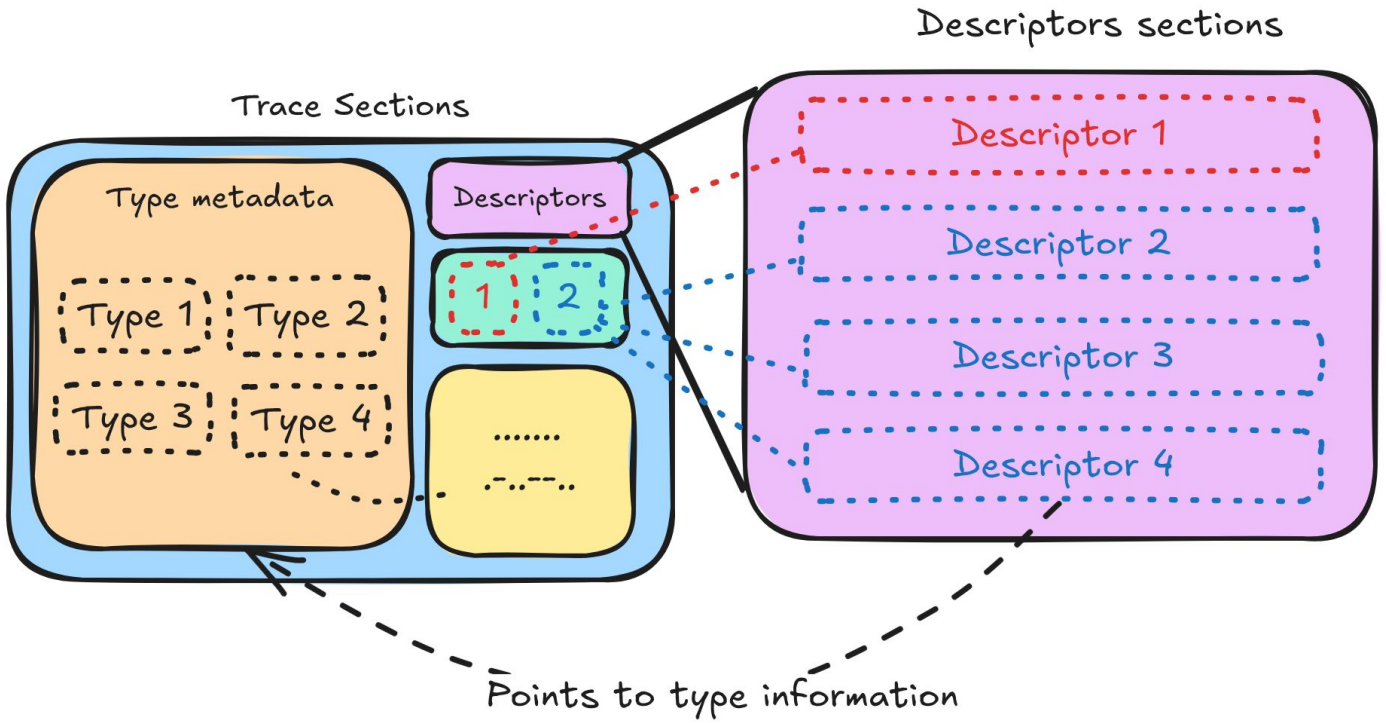

WEKA

# What defines a trace?

## Descriptor ID and type information data

Each log entry template is sort of unique, as it includes the `__LINE__`, `__FUNCTION__`, `__MODULE__` in the template arguments.

This template, then generates a descriptor entry in the sections, aligned by a power of 2 size, so then, at link-time, we can get the log entry ID by subtracting and dividing `(descriptor_instance - __start_section) / alignment_size`.





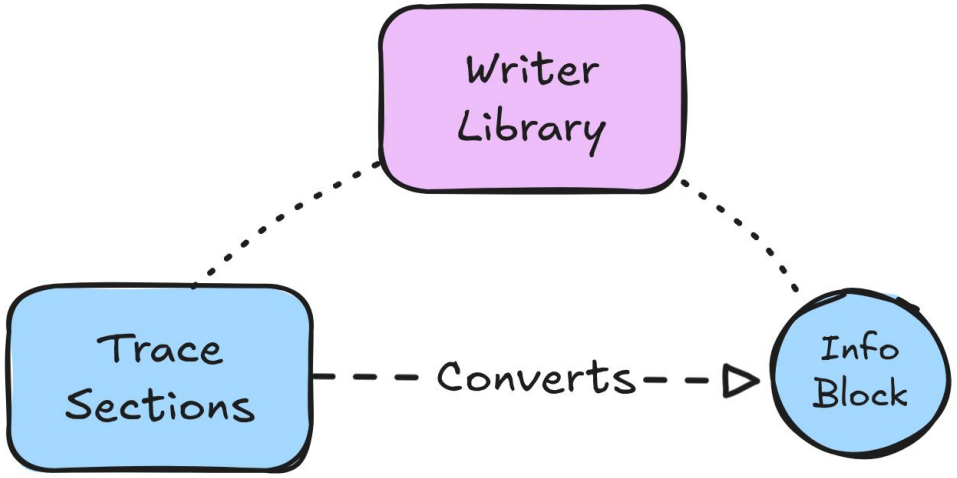

WEKA

# What defines a trace?

## Self-describing trace info block

Now that we have all section data generated by the compiler and arranged by the linker, we need to convert these sections into a self-describing file. We call it trace info block.

The info block contains a copy of the sections as is, but with addresses that are relative to the block itself rather than relative to the binary or the DSO base address.

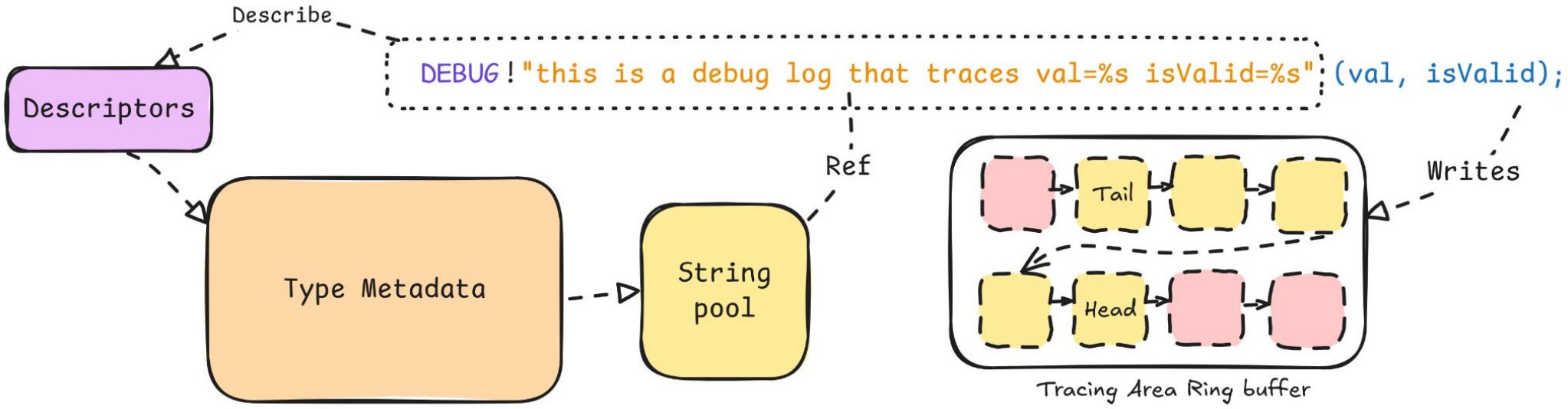





WEKA

# What defines a trace?

## Overview of the writer sections and structures





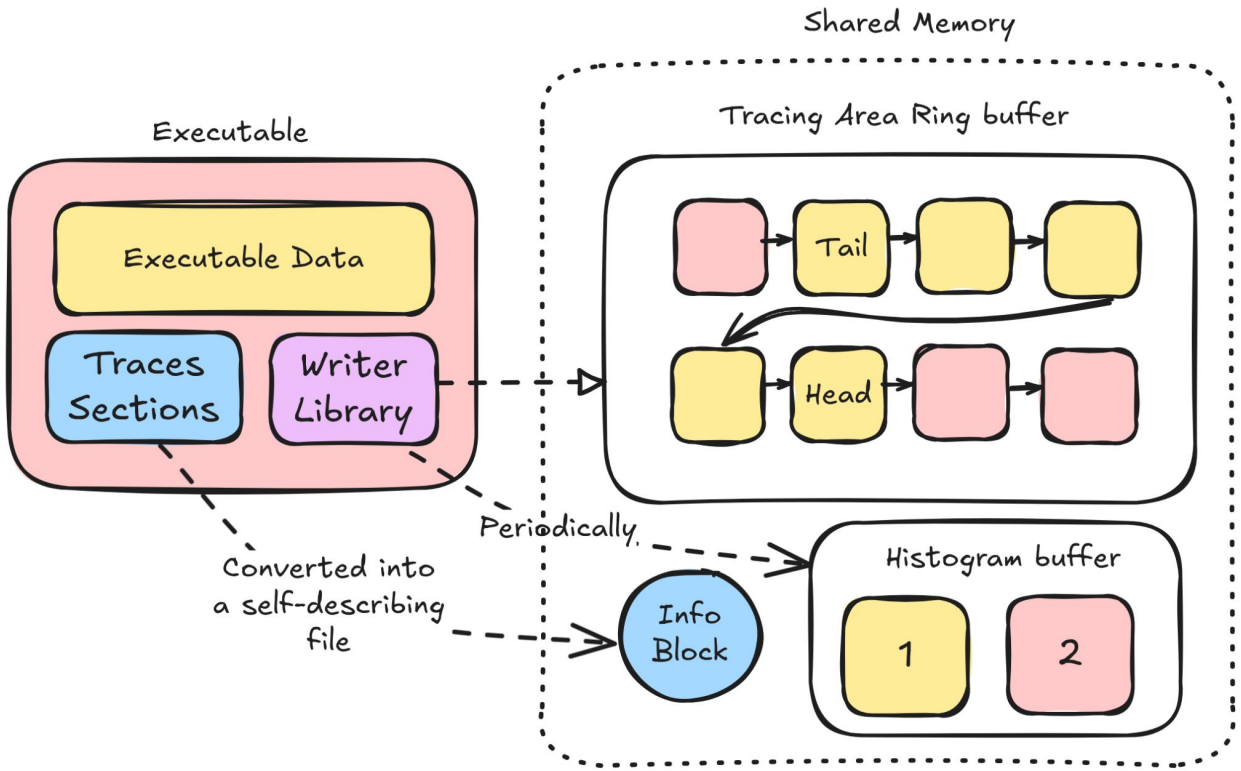

WEKA

# Design structure

## Writing: Runtime flow

At runtime, two things happen:

- When the binary start, the writer library will convert the trace sections into a self-describing file by copying and rewriting the binary relative addresses into block relative addresses.
- When a trace entry is called, the writer library will write the entry header and blit the runtime arguments of the trace, packed (with alignment of 1) into the so called tracing area. When it writes to the tracing area it also hit the histogram by incrementing a counter indexed by the trace ID.






WEKA

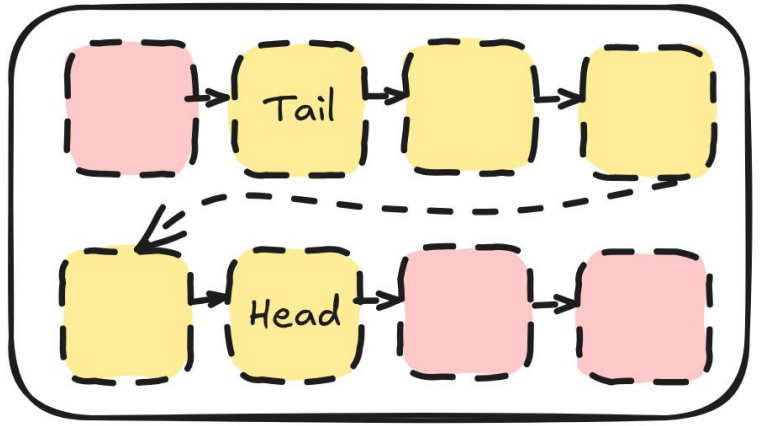
# Design structure

## Writing: Tracing Area

The tracing area is a lockless double indexed ring buffer, that lives in the shared memory and is divided into several chunks, where the data is read and written into.

The writer library controls the head index and the dumper controls the tail index.

This index is double indexed power of 2 value so each component can efficiently and independently check if the ring buffer is filled up or not, without locking the entire state of the ring buffer, assuming only memory atomicity between them.



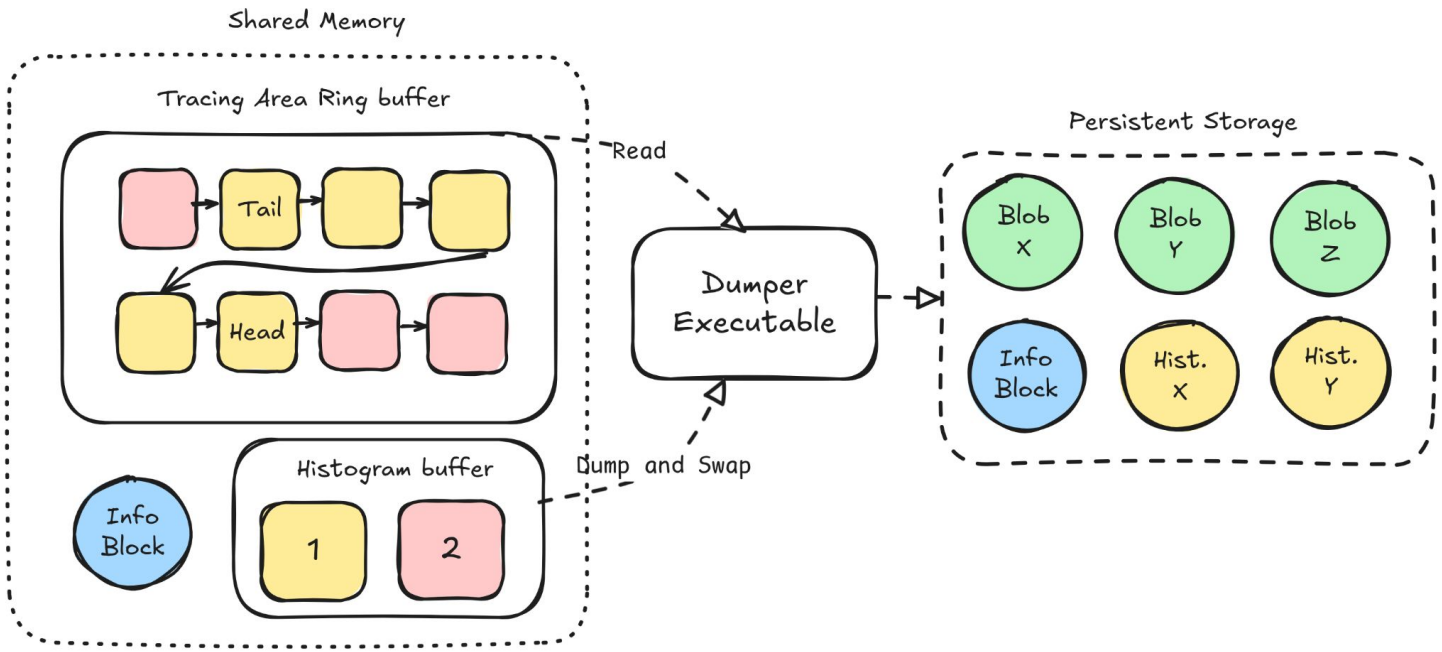
Tracing Area Ring buffer

# Design structure

## Writing: Dumper

The dumper will:

- Move any found info block from the shared memory to persistent storage.
- Dump the traces chunks from the tail of the tracing area into a persistent storage blob.
- Dump and swap the histogram buffers periodically into persistent storage (usually per minute).





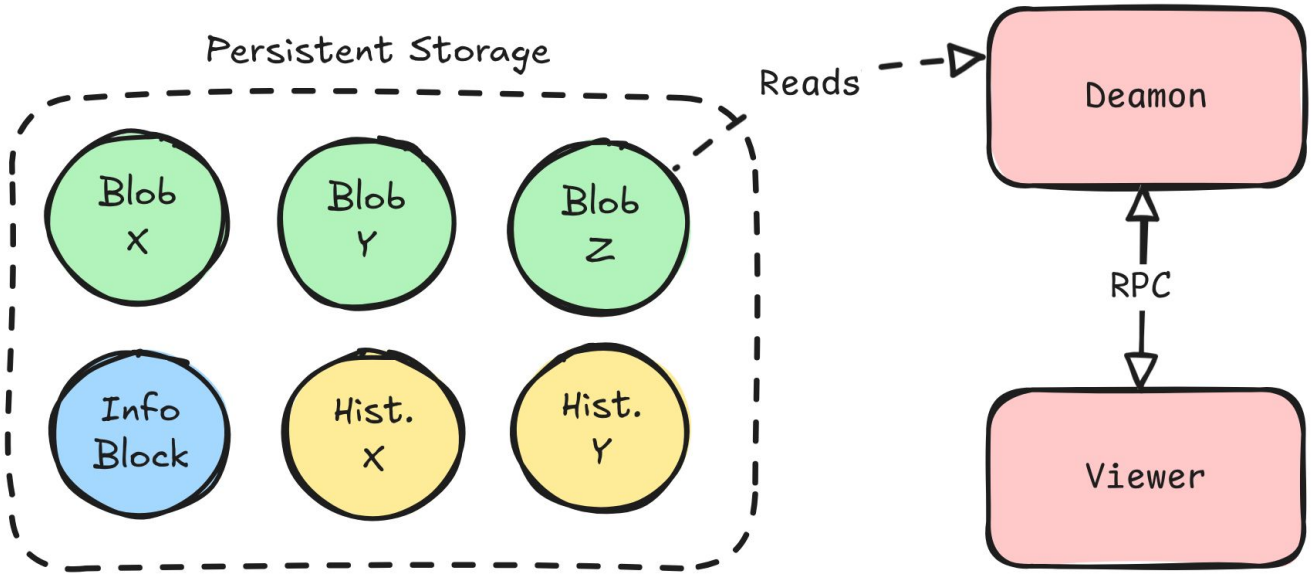

WEKA

# Design structure

**Reading: Daemon and Viewer**

The daemon and viewer is what we use to read traces.

- Daemon provides an RPC service that has the entire persistent storage context
- Viewer requests the daemon with a query, and it returns a partial result




This architecture is useful to read millions of traces over the network, as the context is deferred to a daemon that sits near the traces, while viewer can be anywhere.

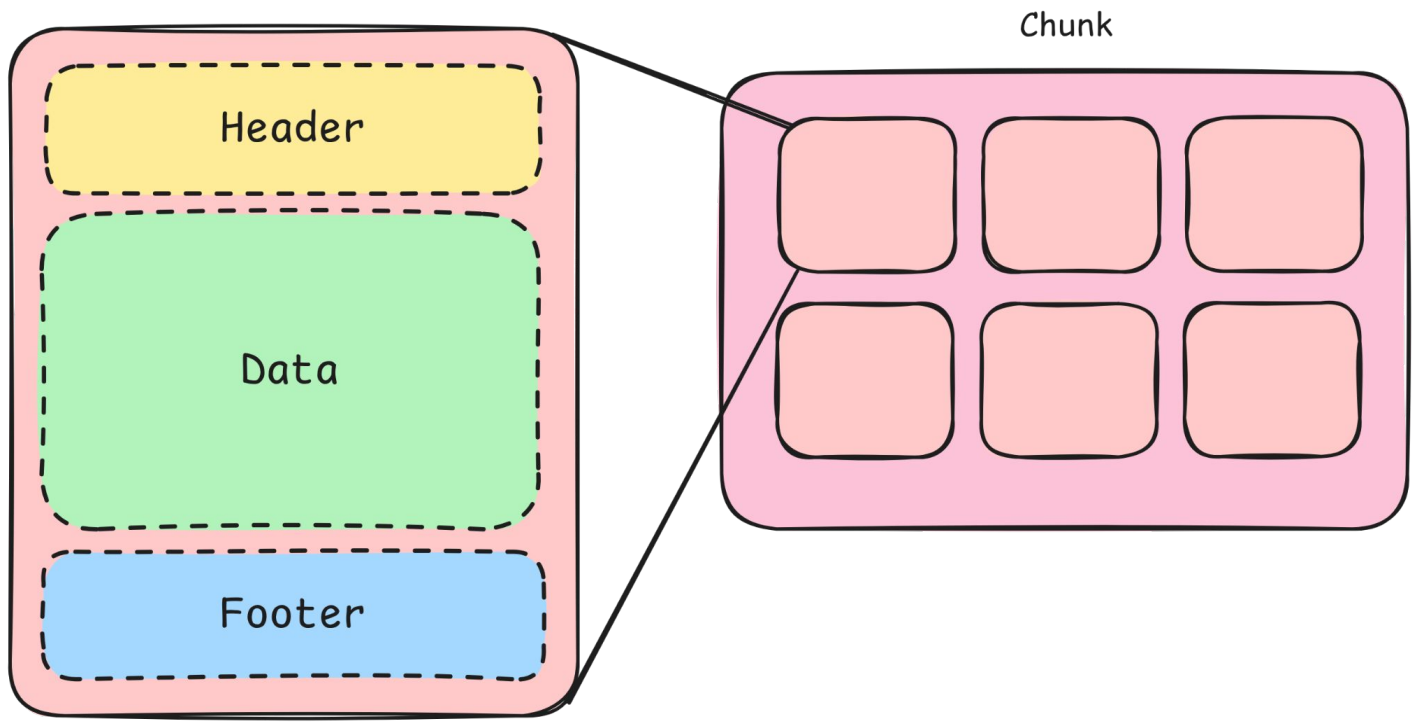
# Design structure

## Reading: Blob data structure

A blob is a type of file that stores traces that is efficient to read from. It's a collection of chunks, which contains a collection of trace entries inside.

For efficiency, we have an header that describes the time window of the contained traces and a footer that has a bitmap.

This bitmap has a bit flipped whenever there's a trace with that descriptor index id in it.



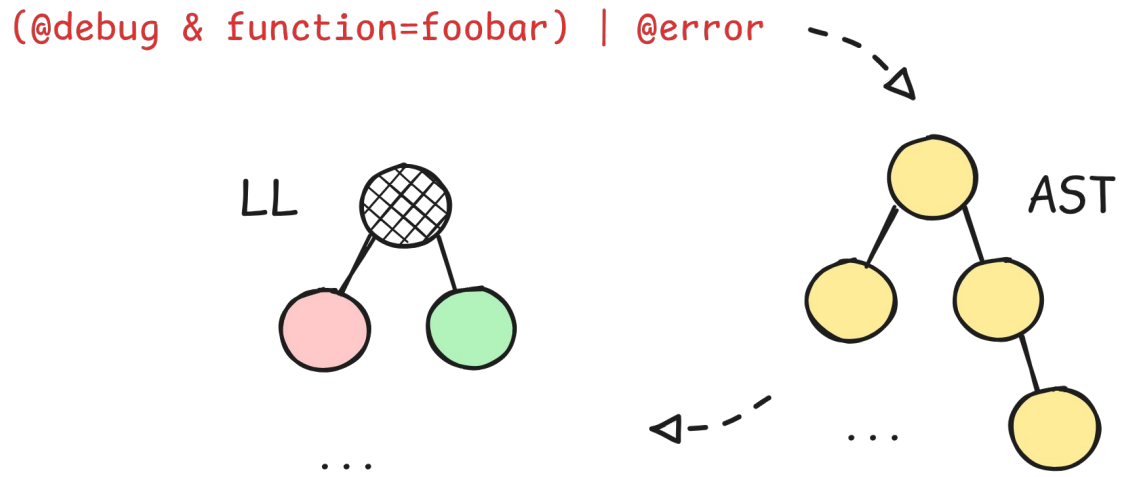
# Design structure

## Reading: Filters

A filter is an expression used to trim out traces that are not relevant to the given query.

We made a small language to expressively filter traces, parsed using pegged D library, and “compiles” to a tree. The structure generated out of this expression is a low-level filter tree composed by very simple nodes used as building blocks to evaluate a given trace entry to a **match** or **not a match** and a bitmap to compare with the chunks.

We make some optimizations so that the resulting tree is the simplest and the smallest.






WEKA

# Design structure

## Reading: Filter examples

You have these three traces:

```
FUNC_ENTER!([“value”])(value /* = ulong(1UL) */);
DEBUG!”tracing string=%s“(str);
WARN!”careful value=%d is bigger than 10“(value);
```

The following filter `(@log | @debug) & $0 == “something”` can be optimized, at the syntax level to: `@log & $0 == “something”` because `@log` is also equivalent to `@debug | @warn | ...`.

When transforming to a low-level filter tree, we have the context of all traces metadata, so, we create a tree with a single “never match” node (equivalent to `false`), for the first and the last trace example and a comparison node for the 2nd trace example (equivalent to simply `$0 == “something”`).

# Demo

# WEKA is Hiring

**These roles open now!**

- Senior Software Engineer, Filesystem
- Senior Software Engineer, Control
- Senior Software Engineer, Platform
- Senior Software Engineer, Protocols

*And more on www.weka.io/dconf*

WEKA®

# Thank You!

 @wekaio

 /wekaio

 @wekaio



WEKA